

Object-Oriented Programming (Classes and Objects)

Study Guide

Mr. Shivkumar Lilhare
Assistant Professor(CSE), PIT
Parul University

1. Classes, Objects, and Object References.....	1
2. Summary of Key Concepts.....	4
3. References.....	4

5.1.1 Classes, Objects, and Object References

◆ 1. What is a Class in Java?

A class is a user-defined data type in Java. It acts as a blueprint for creating objects (instances). It encapsulates data members (fields) and methods (functions) to define the structure and behavior of the objects.

🔗 Syntax of a Class:

```
class ClassName {  
    // Fields (attributes or properties)  
    datatype fieldName;  
  
    // Methods (functions or behaviors)  
    returnType methodName(parameters) {  
        // method body  
    }  
}
```

✓ Example:

```
class Student {  
    int id;          // Field  
    String name;  
  
    void display() { // Method  
        System.out.println(id + " " + name);  
    }  
}
```

◆ Properties of a Class in Java:

1. **Blueprint:**

A class defines the structure (fields) and behavior (methods) of objects.

2. **Does Not Occupy Memory (until instantiated):**

Declaring a class doesn't allocate memory until an object is created.

3. **Can Contain:**

- Fields (variables)
- Methods
- Constructors
- Blocks (static & instance)
- Nested classes
- Interfaces

4. **Access Modifiers:**

Class members can have different access levels: private, public, protected, or default.

◆ **2. What is an Object?**

An **object** is a real-world entity created from a class. It has **state** (data) and **behavior** (methods). Memory is allocated when an object is created using the new keyword.

🧠 **Syntax:**

```
ClassName objRef = new ClassName();
```

✓ **Example:**

```
Student s1 = new Student(); // Object creation  
s1.id = 101;  
s1.name = "John";  
s1.display();
```

◆ **1. Properties of an Object in Java:**

1. **Instance of a Class:**

An object is created from a class using the new keyword.

2. **Has Identity:**

Each object has a unique reference in memory.

3. **Has State and Behavior:**

- **State:** Values of instance variables.
- **Behavior:** What the object can do (via methods).

4. **Memory Allocation:**

Stored in the **heap** memory. References are stored in the **stack**.

5. **Can Access Members of Class:**

Accessed using the **dot operator** (object.method() or object.field).

6. **Can Be Referenced by Multiple Variables:**

Multiple references can point to the same object.

7. **Nullable:**

Object references can be set to null.

◆ 2. Memory Allocation and Object Lifecycle

- When you use new, Java:
 - Allocates memory in the heap.
 - Calls the class constructor.
 - Returns the memory address (reference) to the reference variable.

Illustration:

Object	Reference Variable
new Student()	s1

Student s1 = new Student();

Here, s1 stores the memory address of the object created in the heap.

◆ 3. Object Reference Variables

- Object reference variables **do not store actual object data**, they store the **reference (memory address)** of the object.

✓ Example:

```
Student s1 = new Student();
Student s2 = s1;
```

- Now both s1 and s2 refer to the **same object** in memory.
- Changes via one reference affect the other.

◆ 4. Accessing Class Members

We use the **dot operator (.)** to access fields and methods of the object.

✓ Example:

```
s1.id = 102;
s1.name = "Alice";
s1.display();
```

◆ 5. Null Reference

If you declare an object reference and don't initialize it, it points to **null**.

```
Student s3 = null;
s3.display(); // Causes NullPointerException
```

➤ Summary of Key Concepts:

- A class is a blueprint or template for creating objects.
- It defines fields (data) and methods (behavior).
- An object is an instance of a class created using the new keyword.
- Each object has:
 - State – represented by instance variables.
 - Behavior – represented by methods.
- Object reference is a variable that holds the memory address of the object.
- Class members are accessed using the dot (.) operator (e.g., obj.method()).
- Multiple references can point to the same object in memory.
- Objects reside in heap memory, while reference variables are stored on the stack.
- Java supports encapsulation through classes by combining data and methods.
- Proper use of classes and objects leads to modular, reusable, and maintainable code.

➤ References:

- Oracle Java Documentation – Classes and Objects
 <https://docs.oracle.com/javase/tutorial/java/javaOO/classes.html>
- GeeksforGeeks – Classes and Objects in Java
 <https://www.geeksforgeeks.org/classes-objects-java/>
- JavaTPoint – Class and Object in Java
 <https://www.javatpoint.com/class-and-object-in-java>
- Book: Java – The Complete Reference by Herbert Schildt

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | GRADE



    
<https://paruluniversity.ac.in/>